

1-Number of Zeros in a Given Array

Started on	Friday, 4 September 2026, 4:28 PM
State	Finished
Completed on	Friday, 4 September 2026, 5:11 PM
Time taken	42 mins 31 secs

Time taken	42 mins 31 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Open block dr

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

- First Line Contains Integer m – Size of array
- Next m lines Contains m numbers – Elements of an array

Output Format

- First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```

1 #include<stdio.h>
2 int firstzero(int arr[],int low,int high){
3     if(low>high)
4         return -1;
5     int mid=low+(high-low)/2;
6     if(arr[mid]==0&&(mid==0||arr[mid-1]==1)){
7         return mid;
8     }
9     if(arr[mid]==1){
10        return firstzero(arr,mid+1,high);
11    }
12    else{
13        return firstzero(arr,low,mid-1);
14    }
15 }
```

```

16 ▾ int main(){
17     int m;
18     scanf("%d",&m);
19     int arr[m];
20 ▾   for(int i=0;i<m;i++){
21         scanf("%d",&arr[i]);
22     }
23     int index=firstzero(arr,0,m-1);
24     if(index==-1)
25         printf("0");
26     else
27         printf("%d",m-index);
28     return 0;
29 }

```

	Input	Expected	Got	
✓	5 1 1 1 0 0	2	2	✓
✓	10 1 1 1 1 1 1 1 1	0	0	✓

	1			
	1			
	1			
	1			
	1			
✓	8	8	8	✓
	0			
	0			
	0			
	0			
	0			
	0			
	0			
	0			
✓	17	2	2	✓
	1			
	1			
	1			
	1			

	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	0			
	0			
Passed all tests! ✓				
Correct				



4-Two Elements sum to x

Started on	Friday, 4 September 2026, 9:59 PM
State	Finished
Completed on	Saturday, 5 September 2026, 10:49 AM
Time taken	12 hours 49 mins
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```

1  #include<stdio.h>
2  int findpair(int arr[],int left,int right,int x)
3  {
4      if(left>=right)
5          return 0;
6      int sum=arr[left]+arr[right];
7      if(sum==x){
8          printf("%d\n",arr[left]);
9          printf("%d\n",arr[right]);
10         return 1;
11     }
12     if(sum<x){
13         return findpair(arr,left+1,right,x);
14     }
15     return findpair(arr,left,right-1,x);
16 }
17
18 int main(){
19     int n,x;
20     scanf("%d",&n);
21     int arr[n];
22     for(int i=0;i<n;i++){
23         scanf("%d",&arr[i]);
24     }
25     scanf("%d",&x);
26     if(!findpair(arr,0,n-1,x)){
27         printf("No\n");
28     }
29     return 0;
30 }
31

```

	Input	Expected	Got	
✓	4	4	4	✓
	2	10	10	
	4			
	8			
	10			

	8			
	10			
	14			
✓	5	No	No	✓
	2			
	4			
	6			
	8			
	10			
	100			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00



3-Finding Floor Value

Started on	Friday, 4 September 2026, 9:34 PM
State	Finished
Completed on	Friday, 4 September 2026, 9:58 PM
Time taken	23 mins 59 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```

1  #include<stdio.h>
2  int findFloor(int arr[],int low,int high,int x){
3      int floor=-1;
4      while(low<=high){
5          int mid=low+(high-low)/2;
6          if(arr[mid]==x){
7              return arr[mid];
8          }
9          else if(arr[mid]<x){
10             floor=arr[mid];
11             low=mid+1;
12         }
13         else{
14             high=mid-1;

```

```

15     }
16 }
17 return floor;
18 }
19 int main(){
20     int n;
21     scanf("%d",&n);
22     int arr[n];
23     for(int i=0;i<n;i++){
24         scanf("%d",&arr[i]);
25     }
26     int x;
27     scanf("%d",&x);
28     int res=findFloor(arr,0,n-1,x);
29     printf("%d",res);
30 }

```

	Input	Expected	Got	
✓	6	2	2	✓
	1			
	2			
	8			
	10			
	12			

	10			
	12			
	19			
	5			
✓	5	85	85	✓
	10			
	22			
	85			
	108			
	129			
	100			
✓	7	9	9	✓
	3			
	5			
	7			
	9			
	11			
	13			
	15			
	10			

Passed all tests! ✓



2-Majority Element

Started on	Friday, 4 September 2026, 5:12 PM
State	Finished
Completed on	Friday, 4 September 2026, 7:05 PM
Time taken	1 hour 53 mins
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: `3`

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: `2`

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-231 <= nums[i] <= 231 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```
1  #include<stdio.h>
2  int countoccurrence(int arr[],int low,int high,int element){
3      int count=0;
4      for(int i=low;i<=high;i++){
5          if(arr[i]==element){
6              count++;
7          }
8      }
9      return count;
10 }
11 int majorityelement(int arr[],int low,int high){
12     if(low==high){
13         return arr[low];
14     }
15     int mid=low+(high-low)/2;
16     int leftmajority=majorityelement(arr,low,mid);
17     int rightmajority=majorityelement(arr,mid+1,high);
18     if(leftmajority>rightmajority){
19         return leftmajority;
20     }
21     else{
22         return rightmajority;
23     }
24 }
25 int main(){
26     int n;
27     scanf("%d",&n);
28     int arr[n];
29     for(int i=0;i<n;i++){
30         scanf("%d",&arr[i]);
31     }
32     int ans=majorityelement(arr,0,n-1);
33     printf("%d\n",ans);
34 }
```

	Input	Expected	Got	
✓	3 3 2 3	3	3	✓

passed all tests! ✓



5-Implementation of Quick Sort

Started on	Saturday, 5 September 2026, 10:49 AM
State	Finished
Completed on	Saturday, 5 September 2026, 12:05 PM
Time taken	1 hour 16 mins
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5	12 34 67 78 98
67 34 12 98 78	

Answer:

```
1 #include<stdio.h>
2 void swap(int *a,int *b){
3     int temp=*a;
4     *a=*b;
5     *b=temp;
6 }
7 int partion(int arr[],int low,int high){
8     int pivot=arr[high];
9     int i=low-1;
10    for(int j=low;j<high;j++){
11        if(arr[j]<=pivot){
12            i++;
13            swap(&arr[i],&arr[j]);
14        }
15    }
16    swap(&arr[i+1],&arr[high]);
17    return i+1;
18 }
19 void quicksort(int arr[],int low,int high){
20     if(low<high){
21         int pi=partion(arr,low,high);
22         quicksort(arr,low,pi-1);
23         quicksort(arr,pi+1,high);
24     }
25 }
26 int main(){
27     int n;
28     scanf("%d",&n);
29     int arr[n];
30     for(int i=0;i<n;i++){
```

```

30  for(int i=0;i<n;i++){
31      scanf("%d",&arr[i]);
32  }
33  quicksort(arr,0,n-1);
34  for(int i=0;i<n;i++){
35      printf("%d",arr[i]);
36      if(i<n-1)
37          printf(" ");
38  }
39  return 0;
40
41  }

```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓